

Remainder in Hashing Methods



Mid-square Method

In **mid-square method**, we first square the item, and then extract some portion of the resulting digits.

For example, if the item were 44, we would first compute $44^2=1,936$. By extracting the middle two digits, 93, and performing the remainder step, we get 5 ($93 \% 11$).

Folding Method

The **folding method** for constructing hash functions begins by dividing the item into equal-size pieces (the **last piece may not be of equal size**). These pieces are then added together to give the resulting hash value.

- For example, if our item was the phone number 436 – 555 - 4601, we would take the digits and divide them into groups of 2 (43, 65, 55, 46, 01) (**Fold Shifting**).
- After the addition, $43 + 65 + 55 + 46 + 01$, we get 210.
- If we assume our **hash table has 11 slots**, then we need to perform the extra step of dividing by 11 and keeping the remainder.
- In this case **$210 \% 11$ is 1**, so the phone number 436 – 555 - 4601 hashes to slot 1.

Figure-10.2: Storing key-value in hash table

10.2.2 Mid-square method

Here we have to take the binary representation of the **key_value**. Then we square these binary numbers and pick out the middle k bits. Now these k bits will be used as hash address. The size of the hash table will be (at least) $2^k - 1$.

Example: Binary form of **key_value** = 1011001, by squaring we get 1111011110001, let $k = 3$, then middle 3 bits = 111 = 7. So, the hash address is 7.

Why do we need to find the remainder after mid-square method and folding method in Hashing? In the 2nd picture they used the binary form before Mid-Square method, but did not find the remainder.

Great question! Let's break it down carefully by comparing both examples and explaining **why** the remainder operation (%) is used **sometimes** and **not** in others:

◆ Purpose of Remainder in Hashing

In **hashing**, we want the final value (the *hash*) to be **within the size of the hash table** (i.e. a valid index). If your hash table has m slots, you must ensure the hash is in the range 0 to $m-1$.

That's where the **modulo operation** ($\% m$) comes in — it ensures the computed value fits inside the table.

✓ Case 1: Decimal Mid-Square or Folding Method (1st image)

Mid-Square:

- Squared value: $44^2 = 1936$
- Middle digits: **93**
- Hash table size: 11 slots
- Final step: $93 \% 11 = 5$ (we do this to fit it into table)

Folding:

- Digits grouped and added: $43 + 65 + 55 + 46 + 01 = 210$
- Hash table size: 11 slots
- Final step: $210 \% 11 = 1$ (to get valid index)

👉 **We need to apply $\% \text{table_size}$** because otherwise, the result (**93** or **210**) may be too large to directly index into the hash table.

✓ Case 2: Binary Mid-Square (2nd image)

Here, it's different:

- Key value: **1011001** (binary of 89)
- Squared: **1111011110001**
- Middle k bits are directly extracted (e.g., **111** $\rightarrow$ 7)

So why **no modulo**?

Because:

- They're directly **extracting a fixed number of bits (k)**, and
- That automatically limits the value between 0 to $2^k - 1$.

👉 This is like an **implicit modulo** by 2^k , because:

- $k = 3 \rightarrow 2^3 = 8$ slots, and $111_2 = 7$ is valid.

✅ In Summary

Case	Output Form	Final Step Needed?	Why
Decimal Mid-Square	Middle decimal digits	Yes, % table size	Output may be too large
Folding Method	Sum of digit groups	Yes, % table size	Output may be too large
Binary Mid-Square	Middle k bits from squared binary	❌ No % needed	Already limits its value to 0 to $2^k - 1$

Let me know if you'd like a diagram or code example to clarify further!



So unless the result of Hashing fits the Hash Table or the array size, I do not need to use modulus?

Exactly! You've understood it well.

Rule of Thumb:

Use the modulus (%) only if your hash value might exceed the size of the hash table.

Let's rephrase it cleanly:

Situation	? Should You Use Modulus?	Why?
The hash result is guaranteed to be within the range of 0 to table_size - 1 (e.g., selecting middle k bits where table size is 2^k)	No	Already within bounds
The hash result could be any number , like from squaring, folding, or adding values	Yes (% table_size)	To restrict hash to valid index range

Example 1: No Modulo Needed

Binary mid-square, using 3 middle bits → value is always in **0–7**, so for 8 slots, no % needed.

Example 2: Modulo Needed

Decimal mid-square or folding, result like **210** or **1936** is way bigger than a table of size **11** → you must do **value % 11**.

Key Principle:

Your hash must produce values within $[0, \text{table_size} - 1]$.

If your method already ensures this — you're good. If not — use %.

Let me know if you want a quick checklist to decide when to use it in real-world code!